

SOB4ES - Unión y estandarización final de los datos

CRISP-ML(Q) Fase 2: Ingeniería de Datos

Este notebook combina las salidas de los dos notebooks anteriores en un único dataset listo para modelado.

Paso	Descripción
1	Configuración y rutas
2	Cargar salidas de notebooks 01 y 02
3	Corregir nombres de columna rotos (snake_case sobre acrónimos)
4	Unir datos locales + online
5	Imputar NaN de las variables online
6	Escalar variables online con el mismo scaler del notebook 01
7	Exportar dataset final combinado

****Nota:** **La x en vx se refiere a la versión/iteración del archivo, esto se aplica a todos los outputs empleados.

Entradas: sob4es_clean_vx.csv, sob4es_model_ready_vx.csv, online_features.csv, scaler.pkl, label_encoders.pkl

Salidas: sob4es_final_clean.csv, sob4es_final_model_ready.csv

0.- Configuración

```
In [7]: import os, sys, warnings, re
import numpy as np
import pandas as pd
import joblib
from sklearn.preprocessing import StandardScaler, LabelEncoder

warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', 60)
pd.set_option('display.float_format', '{:.4f}'.format)

OUT_DIR = 'output/'

os.makedirs(OUT_DIR, exist_ok=True)

# Rutas de entrada
CLEAN_PATH = OUT_DIR + 'sob4es_clean_v14.csv'
MODEL_PATH = OUT_DIR + 'sob4es_model_ready_v14.csv'
ONLINE_PATH = OUT_DIR + 'online_features_v3.csv'
SCALER_PATH = OUT_DIR + 'scaler.pkl'
ENCODER_PATH = OUT_DIR + 'label_encoders.pkl'

for nombre, ruta in [
    ('CLEAN_PATH', CLEAN_PATH),
    ('MODEL_PATH', MODEL_PATH),
    ('ONLINE_PATH', ONLINE_PATH),
    ('SCALER_PATH', SCALER_PATH),
    ('ENCODER_PATH', ENCODER_PATH),
]:
    existe = os.path.exists(ruta)
    print(f' {nombre:15s}: {ruta} {"Encontrado..." if existe else "[ERROR] Archivo no encontrado, por favor revisa las rutas..."}')

CLEAN_PATH : output/sob4es_clean_v14.csv Encontrado...
MODEL_PATH : output/sob4es_model_ready_v14.csv Encontrado...
ONLINE_PATH : output/online_features_v3.csv Encontrado...
SCALER_PATH : output/scaler.pkl Encontrado...
ENCODER_PATH : output/label_encoders.pkl Encontrado...
```

1.- Cargar Salidas de los Notebooks Anteriores

```
In [8]: df_clean = pd.read_csv(CLEAN_PATH)
df_model = pd.read_csv(MODEL_PATH)
df_online = pd.read_csv(ONLINE_PATH)

print(f'clean      : {df_clean.shape}')
print(f'model_ready: {df_model.shape}')
print(f'online       : {df_online.shape}')
print()
print('Online columns:', list(df_online.columns))

clean      : (428, 78)
model_ready: (428, 65)
online     : (428, 7)

Online columns: ['site_id', 'gee_temp_media_c', 'gee_humedad_rel_pct', 'gee_ndvi_verano', 'dem_elevacion_m', 'dem_pendiente_deg', 'dem_orientacion_deg']
```

2.- Corregir Nombres de Columna Rotos

La función `estandarizar_nombre()` del notebook 01 aplicó snake_case sobre columnas que ya eran acrónimos en mayúsculas (ej. `SITE_ID`, `BACTERIA_SHANNON`, `CN`). El resultado: cada letra separada por guiones bajos (`s_i_t_e_i_d`, `b_a_c_t_e_r_i_a`...).

Se corrigen aquí con un mapeo explícito antes de hacer cualquier unión.

```
In [ ]: assert 'site_id' in df_clean.columns, 'site_id no encontrado en clean'
assert 'site_id' in df_model.columns, 'site_id no encontrado en model_ready'
assert 'site_id' in df_online.columns, 'site_id no encontrado en online'

# Verificar que no quedan nombres rotos del patrón letra_letra_letra
broken = [c for c in df_clean.columns
          if re.search(r'(?![a-z])([a-z])([a-z])([a-z])', c)]
if broken:
    print(f'[WARN] Columnas con patrón roto detectadas: {broken}')
    print(' → Añadir entradas al RENAME_MAP si las versiones no coinciden.')
else:
    print(f'Nombres de columna verificados ({df_clean.shape[1]} cols en clean)')
print(f'site_id: {df_clean["site_id"].nunique()} sitios únicos en clean')

Nombres de columna verificados (78 cols en clean) ✓
site_id: 428 sitios únicos en clean
```

3.- Unir Datos Locales + Online

```
In [10]: # Unir clean + online por SITE_ID
df_full = df_clean.merge(df_online, on='site_id', how='left', suffixes=('', '_online'))
```

```

assert len(df_full) == len(df_clean), f'Pérdida de filas: {len(df_clean)} → {len(df_full)}'
print(f'Clean : {df_clean.shape}')
print(f'Online : {df_online.shape}')
print(f'Unido : {df_full.shape}')
print()

# Cobertura de variables online
online_feat_cols = [c for c in df_online.columns if c != 'site_id']
cobertura_online = (
    df_full[online_feat_cols].notna().sum() / len(df_full) * 100
).round(1)
print('Cobertura variables online:')
for col, pct in cobertura_online.items():
    print(f'    {col:30s} {pct:.1f}%')

```

```

Clean : (428, 78)
Online : (428, 7)
Unido : (428, 84)

```

```

Cobertura variables online:
gee_temp_media_C          99.8%
gee_humedad_rel_pct       99.8%
gee_ndvi_verano           97.9%
dem_elevacion_m           99.1%
dem_pendiente_deg         99.1%
dem_orientacion_deg       99.1%

```

```

In [11]: assert 'site_id' in df_model.columns, \
         'site_id no encontrado en model_ready, verificar que data-prep.ipynb es la versión correcta...'

print(f'site_id en model_ready: {df_model["site_id"].nunique()} sitios únicos...')
print(f'Shape model_ready: {df_model.shape}')

site_id en model_ready: 428 sitios únicos...
Shape model_ready: (428, 65)

```

4.- Imputar NaN en Variables Online

Misma estrategia que en el notebook 01:

- Variables continuas < 5% NaN: mediana
- Variables continuas 5-50% NaN: columna `_was_missing` + mediana
- > 50% NaN → eliminar

```

In [12]: # Solo aplicar a las columnas online (prefijos gee_, cds_, dem_)
online_num_cols = [c for c in online_feat_cols
                  if df_full[c].dtype in [np.float64, np.float32, np.int64, np.int32]]

miss_pct = df_full[online_num_cols].isnull().mean() * 100

# Eliminar columnas > 50% NaN
drop_online = miss_pct[miss_pct > 50].index.tolist()
if drop_online:
    print(f'Eliminando {len(drop_online)} columnas online con >50% NaN: {drop_online}')
    df_full = df_full.drop(columns=drop_online)
    online_num_cols = [c for c in online_num_cols if c not in drop_online]
    miss_pct = miss_pct.drop(index=drop_online)

# Indicadores _was_missing para columnas 5-50% NaN
cols_indicator = miss_pct[(miss_pct ≥ 5) & (miss_pct ≤ 50)].index.tolist()
for col in cols_indicator:
    df_full[col + '_was_missing'] = df_full[col].isna().astype(int)
print(f'Indicadores _was_missing añadidos para variables online: {len(cols_indicator)}')

# Imputar con mediana
medians_online = df_full[online_num_cols].median()
df_full[online_num_cols] = df_full[online_num_cols].fillna(medians_online)

remaining = df_full[online_num_cols].isnull().sum().sum()
print(f'NaN restantes en variables online: {remaining}')
print(f'Shape tras imputación: {df_full.shape}')

```

```

Indicadores _was_missing añadidos para variables online: 0
NaN restantes en variables online: 0
Shape tras imputación: (428, 84)

```

5.- Escalar Variables Online

Las variables online (`gee_*`, `cds_*`, `dem_*`) no estaban en el dataset cuando se ajustó el `scaler.pkl` del notebook 01.

Se ajusta un scaler **nuevo** solo para estas columnas y se guarda por separado para poder aplicarlo en inferencia.

```

In [13]: # Columnas online a escalar (excluir _was_missing, son binarias)
online_scale_cols = [
    c for c in online_num_cols
    if not c.endswith('_was_missing')
]

scaler_online = StandardScaler()
scaled_online = scaler_online.fit_transform(df_full[online_scale_cols])
df_scaled_online = pd.DataFrame(
    scaled_online,
    columns=[c + '_z' for c in online_scale_cols],
    index=df_full.index
)

joblib.dump(scaler_online, OUT_DIR + 'scaler_online.pkl')
print(f'Escaladas {len(online_scale_cols)} variables online')
print(f'Scaler guardado en {OUT_DIR}scaler_online.pkl')
print(f'Columnas escaladas: {list(df_scaled_online.columns)}')

Escaladas 6 variables online
Scaler guardado en output/scaler_online.pkl
Columnas escaladas: ['gee_temp_media_C_z', 'gee_humedad_rel_pct_z', 'gee_ndvi_verano_z', 'dem_elevacion_m_z', 'dem_pendiente_deg_z', 'dem_orientacion_deg_z']

```

6.- Ensamblar y Exportar Dataset Final

```

In [14]: # 1. Dataset final limpio (legible, escala original)
df_final_clean = df_full.copy()

# Reorganizamos los datos en bloques lógicos (todos los nombres en snake_case)
bloque_geo = ['site_id', 'country', 'pedoclimatic_region', 'site_locality',
              'latitude', 'longitude', 'sampling_date']
bloque_desc = ['soil_type', 'land_use_type', 'land_use_intensity', 'dominant_vegetation']

```

```

cols_totales = list(df_final_clean.columns)

# Abióticos y fisicoquímica del suelo (Ground Truth de campo)
cols_abiotic = [c for c in cols_totales if c in {
    'total_plant_cover', 'bulk_density', 'soil_moisture', 'aggregate_stability',
    'clay_content', 'silt_content', 'sand_content',
    'soil_ph', 'plot_total_c', 'plot_total_organic_c', 'plot_total_n',
    'p', 'k', 'as', 'cu', 'mo', 'ni', 'pb', 'zn'
}]

# Índices globales de diversidad alfa (todos en snake_case tras normalización)
cols_shannon = [c for c in cols_totales if 'shannon' in c.lower()]

# Comunidades de fauna
cols_fauna = [c for c in cols_totales
    if c.startswith(('macro_', 'earthworm_', 'orib_', 'meso_', 'coll_'))
    and c not in cols_shannon]

# Microbioma y secuenciación molecular
cols_micro = [c for c in cols_totales
    if c.startswith(('bac_', 'fun_', 'euk_', 'oomy_', 'cerc_'))
    and c not in cols_shannon]

# Teledetección y DEM
cols_gee_dem = [c for c in cols_totales if c.startswith(('gee_', 'cds_', 'dem_'))]

# Proxies espaciales EU (sin _enc - ya no existen en clean)
cols_eu = [c for c in cols_totales if c.startswith('eu_')]

# Meta-indicadores
cols_meta = ['outlier_flag'] if 'outlier_flag' in cols_totales else []

# Ensamblar el nuevo ordenamiento lógico
nuevo_orden_clean = (
    bloque_geo + bloque_desc + cols_abiotic + cols_shannon +
    cols_fauna + cols_micro + cols_gee_dem + cols_eu + cols_meta
)

# Solo columnas existentes, sin duplicados
nuevo_orden_clean = [c for c in dict.fromkeys(nuevo_orden_clean) if c in df_final_clean.columns]

# Columnas huérfanas al final
columnas_huerfanas_clean = [c for c in df_final_clean.columns if c not in nuevo_orden_clean]
if columnas_huerfanas_clean:
    nuevo_orden_clean += columnas_huerfanas_clean

df_final_clean = df_final_clean[nuevo_orden_clean]

final_clean_path = OUT_DIR + 'sob4es_final_clean.csv'
df_final_clean.to_csv(final_clean_path, index=False)
print(f'{final_clean_path} done...')
print(f'Tamaño: {df_final_clean.shape[0]} sitios × {df_final_clean.shape[1]} columnas')

```

output/sob4es_final_clean.csv done...
Tamaño: 428 sitios × 84 columnas

```

In [15]: # 2. Dataset final listo para modelo: solo _z + outlier_flag (sin _enc)

df_model_final = (
    df_model
    .merge(
        pd.concat([df_full[['site_id']], df_scaled_online], axis=1),
        on='site_id', how='left'
    )
)

assert len(df_model_final) == len(df_model), \
    f'Explosión de filas: {len(df_model)} → {len(df_model_final)} - comprobar site_ids duplicados'

# Columnas de salida: IDs + outlier_flag + todas las _z (campo + online)
id_cols = ['site_id', 'latitude', 'longitude']
flag_col = ['outlier_flag'] if 'outlier_flag' in df_model_final.columns else []
z_cols = [c for c in df_model_final.columns if c.endswith('_z')]

# Ordenar _z siguiendo el mismo orden lógico que nuevo_orden_clean
z_ordered = []
for c in nuevo_orden_clean:
    if f"{c}_z" in z_cols:
        z_ordered.append(f"{c}_z")
# Añadir _z de online que no tengan contraparte en clean (gee_, cds_, dem_)
z_ordered += [c for c in z_cols if c not in z_ordered]

nuevo_orden_model = id_cols + flag_col + z_ordered

# Purgar _was_missing y _enc residuales
nuevo_orden_model = [c for c in nuevo_orden_model
    if not c.endswith('_was_missing') and not c.endswith('_enc')]

# Columnas huérfanas (por si acaso)
huerfanas_model = [c for c in df_model_final.columns
    if c not in nuevo_orden_model
    and not c.endswith('_was_missing')
    and not c.endswith('_enc')]
if huerfanas_model:
    nuevo_orden_model += huerfanas_model

df_model_final = df_model_final[[c for c in nuevo_orden_model if c in df_model_final.columns]]

final_model_path = OUT_DIR + 'sob4es_final_model_ready.csv'
df_model_final.to_csv(final_model_path, index=False)
print(f'{final_model_path} done...')
print(f'Tamaño: {df_model_final.shape[0]} sitios × {df_model_final.shape[1]} columnas')

z_cols_out = [c for c in df_model_final.columns if c.endswith('_z')]
enc_cols_out = [c for c in df_model_final.columns if c.endswith('_enc')]
wm_cols_out = [c for c in df_model_final.columns if c.endswith('_was_missing')]
print(f'Desglose: {len(z_cols_out)} _z | {len(enc_cols_out)} _enc | {len(wm_cols_out)} _was_missing')

output/sob4es_final_model_ready.csv done...
Tamaño: 428 sitios × 71 columnas
Desglose: 67 _z | 0 _enc | 0 _was_missing

```

7.- Resumen Final

```

In [16]: def inferir_fuente(col):
    if col.startswith('gee_'):
        return 'GEE (online)'
    if col.startswith('cds_'):
        return 'CDS (online)'
    if col.startswith('dem_'):
        return 'DEM (online)'
    if col.startswith('eu_'):
        return 'Raster EU'
    if col.startswith('macro_'):
        return 'Macrofauna'

```

```

if col.startswith('orib_'):      return 'Oribátida'
if col.startswith('meso_'):      return 'Mesostigmata'
if col.startswith('coll_'):      return 'Colémbolos'
if col.startswith('bac_'):       return 'Bacterias (16S)'
if col.startswith('fun_'):       return 'Hongos (ITS)'
if col.startswith('euk_'):       return 'Eucariotas (18S)'
if col.startswith('oomy_'):      return 'Oomycetes'
if col.startswith('cerc_'):      return 'Cercozoa'
if col.startswith('plot_'):      return 'Abiótico (parcela)'
if col in {'clay_content', 'silt_content', 'sand_content',
           'aggregate_stability', 'bulk_density', 'soil_moisture'}:
    return 'Abiótico (físico)'
if col in {'as', 'cu', 'k', 'mo', 'ni', 'p', 'pb', 'zn', 'soil_ph'}:
    return 'Abiótico (químico)'
if any(x in col for x in ('shannon', 'richness', 'abundance', 'asv_', 'reads')):
    return 'Diversidad alfa'
if col in {'site_id', 'latitude', 'longitude', 'country',
           'pedoclimatic_region', 'sampling_date', 'site_locality',
           'soil_type', 'land_use_type', 'land_use_intensity',
           'dominant_vegetation', 'total_plant_cover'}:
    return 'Metadatos sitio'
if col.endswith('_z'):           return 'Escalado (z)'
if col.endswith('_enc'):         return 'Codificado (enc)'
if col.endswith('_was_missing'): return 'Indicador NaN'
return 'Otro'

```

```

catalogo = pd.DataFrame({
    'dtype'      : df_final_clean.dtypes,
    'n_nulos'    : df_final_clean.isnull().sum(),
    'n_unicos'   : df_final_clean.nunique(),
    'fuente'     : [inferir_fuente(c) for c in df_final_clean.columns],
})

```

```

print('  Columnas por fuente  ')
print(catalogo.groupby('fuente').size().sort_values(ascending=False)
      .rename('n_columnas').to_string())
print()
print('  Resumen final  ')
print(f'Sitios      : {df_final_clean.shape[0]}')
print(f'Columnas (clean) : {df_final_clean.shape[1]}')
print(f'Columnas (model_ready): {df_model_final.shape[1]}')
print(f'NaN totales (clean) : {df_final_clean.isnull().sum().sum()}')
print(f'NaN totales (model) : {df_model_final.isnull().sum().sum()}')

```

```

Columnas por fuente
fuente
Raster EU      15
Metadatos sitio 12
Abiótico (químico) 9
Abiótico (físico) 6
Diversidad alfa 4
Colémbolos     4
Abiótico (parcela) 3
DEM (online)   3
Cercozoa       3
Bacterias (16S) 3
GEE (online)   3
Eucariotas (18S) 3
Macrofauna     3
Hongos (ITS)   3
Mesostigmata   3
Oomycetes      3
Oribátida      3
Otro           1

Resumen final
Sitios      : 428
Columnas (clean) : 84
Columnas (model_ready): 71
NaN totales (clean) : 0
NaN totales (model) : 0

```